Final Project Proposal

Group members: Karina Yaheli Puente , Colin Mitchell , Abhinav Jain , Akshaya Agrawal

Introduction:

A scavenger hunt is a fun, team oriented, activity in which a team of organisers create a list of objects that need to be found by the participating teams. Traditionally, the list items must be found sequentially and the team that finishes locating all the items is the victor. However, while this is a fun game for kids, it is an incredibly difficult activity for a robot to participate in. The goal of our project is to enable Fetch to help the team by identifying an object, moving to the object, and picking it up. Although this would only cover a single line item on the list, it is the first step in robot scavenger hunt participation. In order for Fetch to properly help the team, it needs to be able to accurately map the space before trying to identify objects and grasp them. In this assignment, we verify that we can make a map of any scene and store it for future use.

Desired Interactions:

Sensing:

- **Hardware Components:**

  We will be using Fetch Robot (Figure 1) as the main component for this project. The idea is to use Fetch's RGB-D camera located at the head and the laser scanner located at the base to map a desired environment. The hand will move its joint to pick up the object. Robot components shown in Figure 2. The user interface will be the monitor, keyboard and mouse. Lastly, we will use a laptop to operate the Fetch robot remotely by SSH into it.

- **Software Components:**

  We will be using the following packages in Fetch:

  - ROS package for Fetch integration: fetch_ros
  - Teleoperate Fetch: teleop_twist_keyboard
  - Build Map: fetch_navigation
  - Save the generated 2D Map: map_server
  - Save the generated Octomap: octomap_server
  - Visualization tool: RViz
  - Simulation tool: Gazebo
  - Python files (see: ROS Functions & Scripts used and what they do)

- **Actions Fetch will take:**

  Subscribe to topics: PointCloud2 and LaserScan. Afterwards use the fetch_navigation ros package. We built a map of the workspace and saved it using the map_server ros package. Mapping this environment raised some issues for us because of the region outside the workspace area. We then generated more refined maps to fix the issue.

  To understand and address the challenges faced for generating an Octomap (We will need this for object segmentation/path planning for fetch arm), we went ahead and built a small octomap

  Basic Actions:

  - Move to a specified point(front end of the table) in the built map
  - Locate the desired object (yellow cube)
  - Pick up object
  - Wave goodbye with humans

- **ROS Functions & Scripts used and what they do:**
  - roslaunch fetch_navigation fetch_nav.launch map_file:=/home/fetch/catkin_ws/src/fetchRobot/map/RawMap_20211110.yaml
    - Starts the move groups for base movement and initializes the map specified by map_file
    - Moves the map such that it overlays in rviz over the robot
    - Changes the origin in map.yaml to move the map such that it overlays in rviz over the robot such that it matches the real world robot and (environment)map setup.
  - Python: nav_alan.py
    - Defines the target goal position in the map and moves the robot to the target
  - rosrun tf tf_echo /map /base_link
    - Used to see the transformation matrices between the map and robot base_link
  - Python: pcl_filter.py
    - Converts a ROS PointCloud2 message to a separate XYZ range images, RGB image
    - Uses cube_detection script to detect a cube
    - Obtains the coordinates of the yellow cube
  - Test_xyz.py
    - Test file for pcl_filter.py
  - Python: pcl_helper.py

        - This file contains helper functions to manipulate the data from the PointCloud2 message received from fetch.
    - roslaunch fetch_moveit_config move_group.launch
        - This is required to start the planner for arm manipulation.
    - Python: move_arm.py
        - Takes the cube center coordinate input from pcl_filter.py
        - Invokes the planner to reach the (x,y,z) coordinates and then closes gripper
        - Tucks arm after grabbing the cube
    - Commands/Steps to move arm with respect to the base link:
        1. roslaunch~/Downloads/projects/introRobotics/fetchRobot_ws/src/fetchRobot/launch/fetch_world_alan.launch
        2. rosrun rviz rviz
        3. rostopic echo /clicked_point
        4. roslaunch fetch_moveit_config move_group.launch
        5. rosrun teleop_twist_keyboard teleop_twist_keyboard.py
        6. Python: move_arm.py
    -
- **Thinking:**
    - A human will place the yellow cube on the table to show human interaction. After placing the cube, Fetch will begin its scavenger hunt. Fetch will move from a desired starting point in the map (Figure 3). Next it will move in front of the table and use its camera to get the coordinate point for the yellow cube. After getting the coordinates, Fetch will decide how close to get to the yellow cube without colliding with the table shown in Figure 4. If Fetch knows it's arm will hit the table, it will decide to stop. Otherwise, Fetch will proceed and pick up the yellow cube, tuck in it's arm with the cube, and lastly turn around and wave goodbye with humans.

Figure 1: Fetch Robot

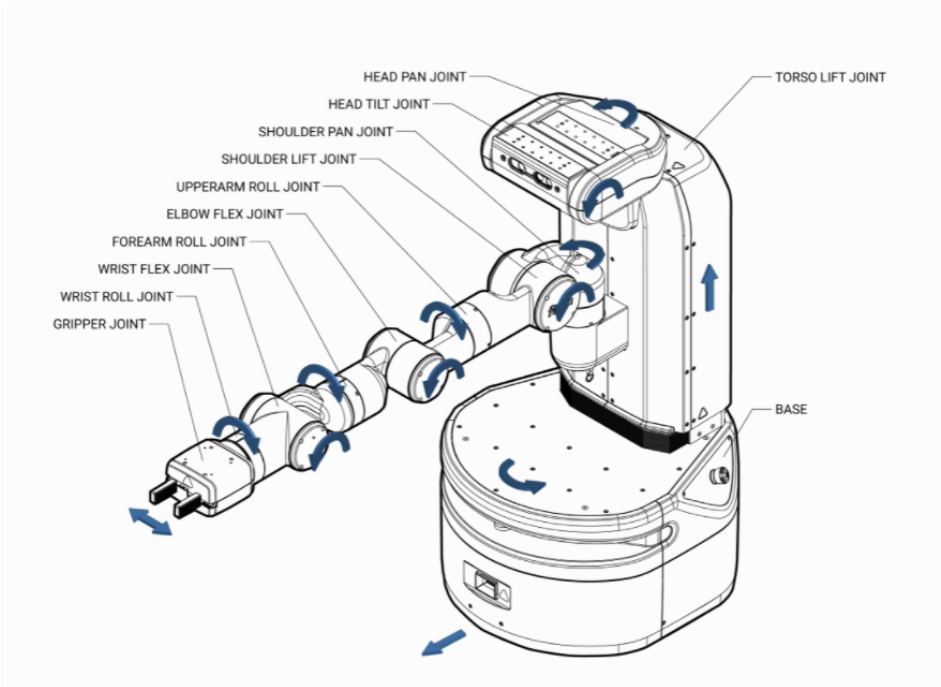

Figure 2: Robot Components (fetchrobot.com)

Figure 3: Mapped environment

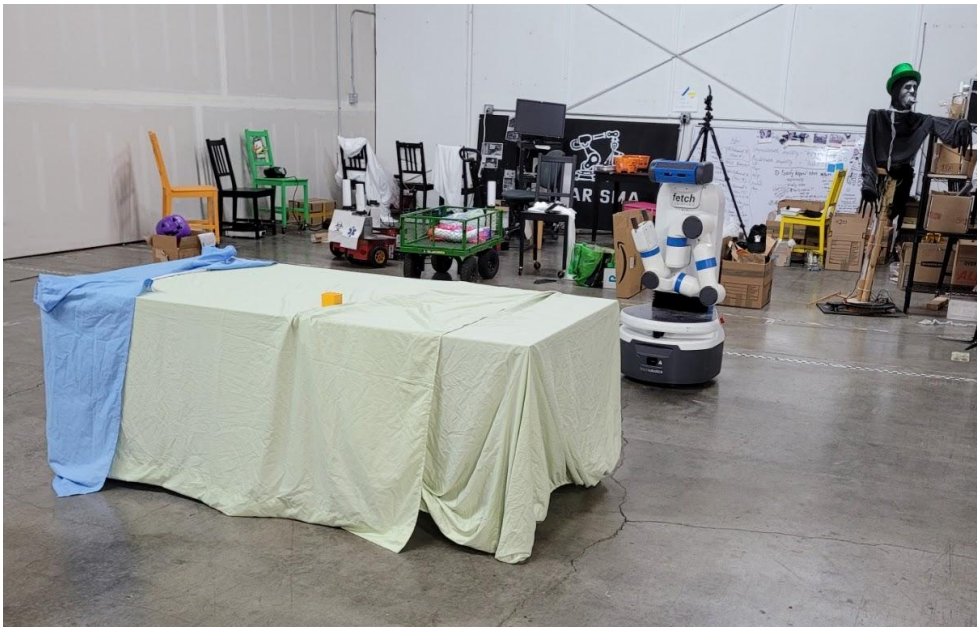

Figure 4: Real world environment